

PNEUMOVISION

软件设计说明书

目录

1	概述	2
1.1	编写目的	2
1.2	文档范围	2
1.3	定义与缩略语	2
1.4	软件架构概述	3
2	设计目标和约束	4
2.1	设计目标	4
2.1.1	功能完整性	4
2.1.2	性能达标	4
2.1.3	可靠性	5
2.1.4	安全性	5
2.1.5	可维护性	5
2.1.6	可扩展性	5
2.2	设计约束	5
2.2.1	技术约束	5
2.2.2	项目约束	6
2.2.3	环境约束	6
2.2.4	合规约束	6
2.2.5	质量约束	6
3	软件架构设计	7
3.1	架构设计	7
3.1.1	架构图及说明	7
3.1.2	架构设计关键点	9
3.2	用户界面设计	14
3.3	接口设计	15
3.3.1	接口设计原则	15
3.3.2	接口访问与认证方式	16
3.3.3	后端核心接口分类	18
3.3.4	AI 推理服务内部接口	20
3.3.5	典型接口调用流程	21
3.3.6	接口安全与异常处理	25
3.4	数据设计	26
3.5	部署设计	27

1 概述

1.1 编写目的

本文档为医疗影像数据管理平台的软件设计说明书，旨在描述系统的整体架构、模块划分、接口定义、数据存储设计及关键技术方案，为后续的编码实现、测试验证及系统部署提供设计依据与技术指导。

本文档的主要目标包括：

- 1. 明确系统的高层架构与技术选型；
- 2. 详细定义各功能模块的职责与交互关系；
- 3. 规范前后端接口契约与数据格式；
- 4. 指导开发团队进行模块化并行开发；
- 5. 为系统维护与后续扩展提供设计参考。
- 6. 本文档的预期读者包括：项目管理人员、系统架构师、开发工程师、测试工程师及质量保证人员。

1.2 文档范围

本文档覆盖医疗影像数据管理平台的整体设计方案，具体内容包括：

- 1. **系统架构设计：**包括总体架构图、模块划分及部署视图；
- 2. **数据库设计：**包括概念模型（ER 图）、物理表结构设计及关键字段说明；
- 3. **接口设计：**包括前后端接口规范、核心 API 定义及数据格式说明；
- 4. **模块详细设计：**包括前端模块划分、后端业务分层及 AI 服务封装设计；
- 5. **部署设计：**包括容器化方案、环境配置及一键启动说明；
- 6. **安全设计：**包括认证授权机制、数据安全与传输加密策略。

1.3 定义与缩略语

术语/缩写	定义与解释
PneumoVision	指本组设计的医疗影像数据管理平台
平台/本平台	指“医疗影像数据管理平台”，即本文档所描述的软件系统。
MVP	最小可行产品（Minimum Viable Product），指包含足够满

	足核心用户需求的最少功能集的产品版本。
DICOM	医学数字成像与通信标准（Digital Imaging and Communications in Medicine），医学影像国际标准格式，文件常以.dcm 为扩展名。
脱敏	对敏感信息（如患者姓名、身份证号）进行处理，使其在不泄露隐私的前提下仍可用于教学或测试。

1.4 软件架构概述

医疗影像数据管理平台采用前后端分离 + Java 业务服务 + Python AI 推理服务的分层架构，整体分为表现层、接入层、业务层、AI 推理层、数据与基础设施层。

1. 表现层

系统前端基于 React + Vite 构建，按角色拆分为登录页、医生端、管理员端和科研端。前端负责页面展示、表单交互、影像预览、检测框可视化、分页检索、任务状态展示和结果导出触发等功能。

2. 接入层

系统通过 Nginx 提供静态资源访问和反向代理能力，将不同前端应用映射到对应访问路径，并将 /api 请求转发至 Spring Boot 后端服务。该层负责统一入口、路径分发、静态资源缓存和跨服务访问屏蔽。

3. 后端业务层

后端基于 Spring Boot 构建 RESTful API，负责用户认证、RBAC 权限控制、病例管理、检查管理、影像上传、DICOM 导入、推理任务调度、结果存储、医生评价、结果导出和日志审计。后端采用 Controller、Service、Repository 分层结构，通过 JWT 实现无状态认证，并在接口层进行角色和数据归属校验。

4. AI 推理层

AI 推理层基于 Python FastAPI 封装本地目标检测模型，当前支持 YOLOv11 与 RT-DETR 两类模型。推理服务负责影像预处理、模型加载、模型推理、结果后处理和统一结果格式返回。系统早期 ResNet50 分类模型仅能输出影像级分类结果，无法定位肺炎区域，因此已升级为目标检测模型，以满足病灶定位、检测框可视化和模型对比需求。

5. 数据与基础设施层

系统采用 MySQL 存储用户、病例、检查、影像、推理任务、推理结果、模型配置和操作日志等结构化数据；影像文件、导出文件和可视化标注图存储于服务器文件系统或对象存储路径；Redis 用于缓存推理结果、提升重复访问速度，并可配合异步任务状态管理；消息队列用于推理任务异步化，避免长时间推理阻塞前端交互。

部署架构

所有服务通过 Docker Compose 进行容器化编排，包含前端容器、后端 API 容器、AI 推理容器及 mysql 数据库容器。开发与测试过程中使用 Postman 进行接口调试，DBeaver 进行数据库管理，Swagger UI 自动生成并展示 API 文档。

版本控制

项目采用 Git 进行版本管理，代码仓库托管于 GitHub，遵循规范的分支策略与提交规范，确保团队协作顺畅与代码可追溯。

核心业务流程

用户上传影像后，发起推理请求；后端创建任务并调用 AI 服务；AI 服务完成检测并返回结果（标签、置信度、检测框）；后端保存结果并返回前端；前端在原图上进行可视化展示。整个过程对用户透明，实现“一键辅助诊断”。

2 设计目标和约束

2.1 设计目标

本系统的架构设计遵循以下核心目标，确保最终交付成果满足教学实践要求与技术规范：

2.1.1 功能完整性

系统应完整覆盖 MVP 定义的七大核心功能模块：账号权限管理、影像数据管理、AI 模型集成、推理任务管理、结果可视化、结果导出、日志审计，形成从上传到导出的业务闭环。

2.1.2 性能达标

单张影像推理响应时间 ≤ 3 秒（不含上传耗时）；

系统可支持 10 ~ 20 个并发请求（教学演示负载）；

主要页面加载时间 ≤ 3 秒（本地网络环境）。

2.1.3 可靠性

关键错误可追踪（具备日志记录与请求 ID）；

AI 服务异常不导致主系统崩溃，支持优雅降级；

推理任务状态准确流转（待处理/成功/失败），支持失败重试或查询。

2.1.4 安全性

敏感数据脱敏展示；

关键操作记录审计日志。

2.1.5 可维护性

代码分层清晰（Controller/Service/Repository）；

提供完整的 API 文档（Swagger UI）；

配置与代码分离，支持多环境部署；

遵循统一的代码规范，便于团队协作。

2.1.6 可扩展性

系统设计应考虑后续功能扩展（如批量处理、多模型对比、异步任务队列）；

核心模块间低耦合，便于独立升级与替换。

2.2 设计约束

2.2.1 技术约束

前端技术栈：React、Vite、JavaScript、CSS，用于构建医生端、管理员端、科研端与登录页；

后端技术栈：Spring Boot、Spring Security、Spring Data JPA/MyBatis、JWT，提供 RESTful API 与权限控制；

AI 技术栈：Python、FastAPI、PyTorch/Ultralytics，封装 YOLOv11 与 RT-DETR 目标检测模型；

数据库：MySQL 作为主数据库，存储用户、病例、影像、任务、结果、模型配置与日志等结构化数据；

缓存与队列：Redis 用于缓存推理结果与提升重复访问速度；消息队列用于推理

任务异步处理；

部署方式：支持 Linux 服务器部署，前端构建后由 Nginx 托管，后端和 AI 服务独立运行或容器化部署；

接口文档：后端使用 Swagger/OpenAPI 生成接口文档，便于前后端联调和测试；

测试工具：使用 Postman/Apifox 进行接口测试，DBeaver 进行数据库管理。

2.2.2 项目约束

开发周期：项目需在 9 周内完成从需求分析到部署验收的全流程；

团队规模：6 人团队，包含项目经理、前端开发、后端开发、AI 开发、测试等角色；

交付成果：需交付可运行系统、完整源码、设计文档、测试报告、部署手册及结项报告；

教学定位：系统定位为教学实验原型，非商业化产品，不追求医院级高可用与全量集成。

2.2.3 环境约束

硬件环境：最低配置要求 2 核 CPU、4GB 内存、20GB 磁盘空间；

软件环境：支持 Linux/Windows 服务器部署，依赖 Docker Engine 20.10+ 及 Docker Compose 2.0+；

数据来源：所有影像数据均来自公开数据集（如 RSNA 肺炎检测挑战赛数据）或完全脱敏的教学数据，不涉及真实患者隐私。

2.2.4 合规约束

数据隐私：涉及影像数据需符合《个人信息保护法》对模拟数据的基本要求，不得包含真实患者隐私信息；

医疗规范：影像处理参考 DICOM 标准（仅格式兼容，不实现全量通信协议）。

2.2.5 质量约束

测试覆盖率：单元测试覆盖率 $\geq 70\%$ ；

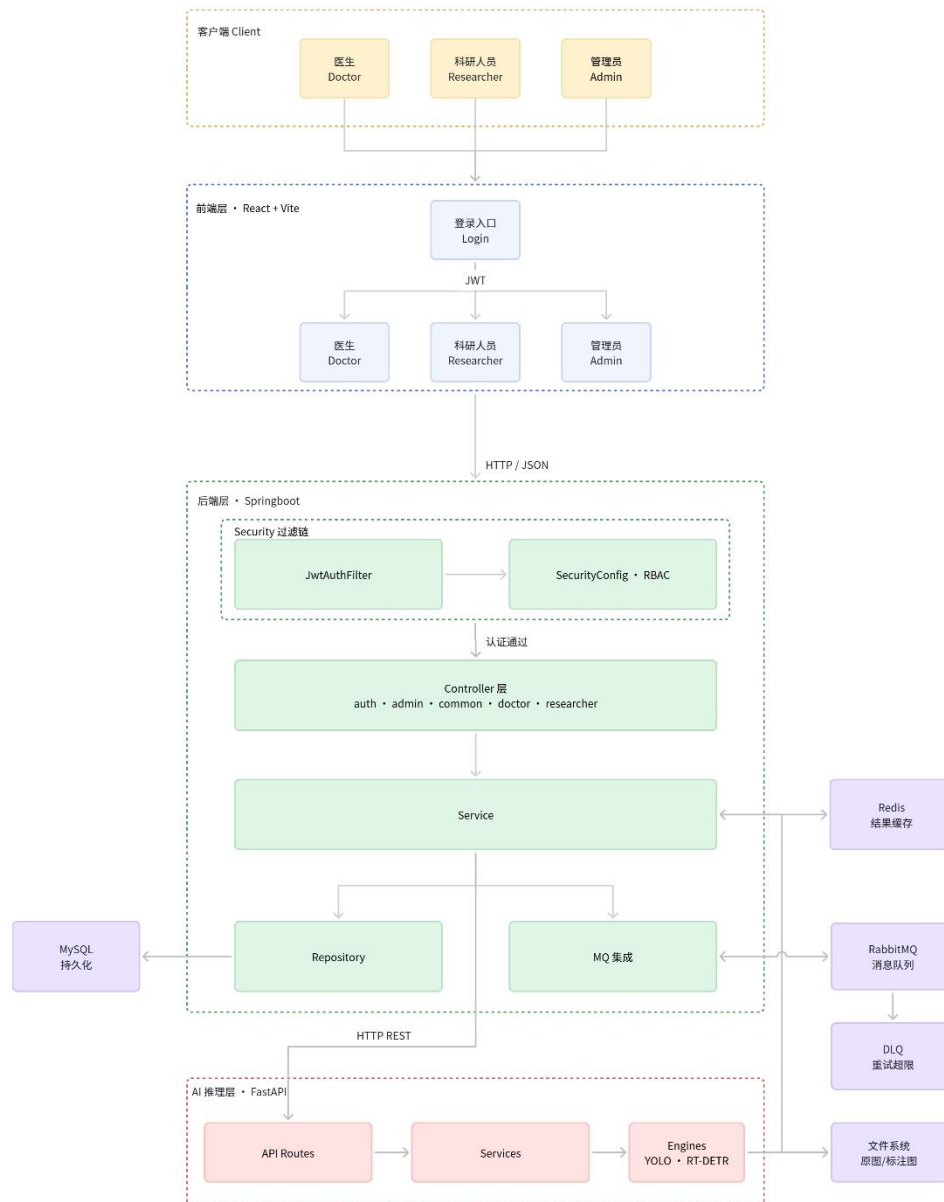
代码规范：前端遵循统一的代码风格，后端遵循 Java 社区通用编码规范；

文档完整性：需提供完整的 README、接口文档、部署说明及用户手册。

3 软件架构设计

3.1 架构设计

3.1.1 架构图及说明



医疗影像数据管理平台采用**前后端分离 + Java 业务服务 + Python AI 推理服务 + 中间件支撑**的分层架构。系统整体自上而下可划分为客户端层、前端展示

层、后端业务层、AI 推理层以及数据与基础设施层，各层职责清晰、相互解耦，便于功能扩展、模型替换和后续维护。

1. 客户端层

客户端层面向系统的三类核心用户：医生、科研人员和系统管理员。用户通过 Web 浏览器访问系统，不同角色登录后进入对应的业务界面。

医生主要负责病例管理、检查管理、影像上传、DICOM 智能导入、发起推理、查看结果、医生评价和结果导出；科研人员主要负责浏览影像数据、查看模型结果、配置模型参数和进行多模型对比；管理员主要负责用户管理、权限配置和日志审计。

2. 前端展示层

前端基于 **React + Vite** 构建，包含统一登录入口以及医生端、科研人员端、管理员端三个角色工作台。用户登录成功后，前端保存后端返回的 JWT，并在后续请求中携带该令牌访问业务接口。

前端主要负责页面展示、表单交互、影像预览、检测框可视化、分页检索、任务状态展示、模型配置操作以及结果导出触发。系统通过角色信息控制前端菜单展示，使不同角色进入系统后看到与其权限匹配的功能模块。

前端与后端之间采用 **HTTP / JSON** 方式通信，接口风格遵循 **RESTful** 设计原则。

3. 后端业务层

后端基于 **Spring Boot** 构建，是系统的核心业务控制中心。后端整体采用 **Controller、Service、Repository** 分层结构，并集成 **Spring Security** 完成认证与授权控制。

在安全过滤链中，**JwtAuthFilter** 负责解析请求中的 JWT，识别当前登录用户身份；**SecurityConfig + RBAC** 负责基于角色控制接口访问权限。认证通过后，请求进入 **Controller** 层，由不同业务模块处理具体请求。

后端 **Controller** 按业务功能划分为 **auth、admin、common、doctor、researcher** 等模块，分别负责登录认证、管理员功能、公共接口、医生端业务和科研端业务。**Service** 层封装核心业务逻辑，包括病例管理、检查管理、影像管理、DICOM 解析与导入、推理任务创建、结果保存、医生评价、结果导出、模型配置和日志审计等。**Repository** 层负责与 **MySQL** 数据库交互，完成结构化数据的持久化存储。

后端同时集成 **Redis** 和 **RabbitMQ**：**Redis** 用于缓存推理结果和提升重复访问速度；**RabbitMQ** 用于推理任务异步化，使前端提交任务后无需长时间等待模型推理完成。

4. AI 推理层

AI 推理层基于 **FastAPI** 独立部署,作为后端业务服务之外的 Python 推理服务。Spring Boot 后端通过 HTTP REST 调用 FastAPI 接口,完成影像推理请求。

AI 推理服务内部划分为 API Routes、Services 和 Engines 三部分。API Routes 负责接收推理请求; Services 负责任务参数处理、图像预处理、结果后处理和统一响应封装; Engines 负责加载和调用具体 AI 模型。

当前系统已从早期 ResNet50 分类模型升级为 **YOLOv11 与 RT-DETR 目标检测模型**。ResNet50 只能判断影像是否疑似肺炎,无法定位病灶区域; YOLOv11 和 RT-DETR 能够返回检测框、类别、置信度和坐标信息,更适合胸部 X 光肺炎区域检测和结果可视化展示。

通过独立 AI 服务设计,系统可以在不影响主业务后端的情况下替换模型、增加新模型或调整推理参数。

5. 数据与基础设施层

数据与基础设施层为系统提供持久化存储、缓存、异步队列和文件管理能力。

MySQL 用于存储系统结构化数据,包括用户、角色、病例、检查、影像、推理任务、推理结果、模型配置、医生评价和操作日志等信息。

Redis 用于存储推理结果缓存。当同一影像、同一模型和相同参数组合下的结果被重复访问时,系统可优先读取缓存,减少重复计算并提升响应速度。

RabbitMQ 用于推理任务消息队列。后端创建推理任务后,将任务投递到队列中,由后台消费者异步处理。对于异常任务,系统可通过 DLQ 死信队列进行失败隔离和重试处理,避免单个异常任务影响整体队列运行。

文件系统用于存储影像原图、DICOM 转换结果、标注结果图、导出文件和 PDF 报告等非结构化文件。数据库中仅保存文件路径、文件类型和关联关系,从而降低数据库存储压力。

3.1.2 架构设计关键点

1. 分层解耦设计

系统采用前端、后端、AI 推理服务和基础设施分层设计。前端专注页面展示和用户交互,后端专注业务逻辑和权限控制, AI 推理服务专注模型加载与推理计算, MySQL、Redis、RabbitMQ 和文件系统分别提供数据存储、缓存、异步处理和文件管理能力。

这种分层设计降低了模块之间的耦合度，使各部分可以独立开发、测试、部署和维护。例如，后续如果需要替换 AI 模型，只需调整 FastAPI 推理服务和模型配置，不需要大规模修改前端和业务后端。

2. 认证授权与 RBAC 权限控制

系统采用 JWT 作为无状态认证机制。用户登录成功后，后端返回 JWT，前端在后续请求中携带该令牌。后端通过 JwtAuthFilter 解析令牌，获取当前用户身份，并结合 Spring Security 完成接口权限校验。

系统基于 RBAC 模型划分管理员、医生、科研人员三类角色。不同角色拥有不同的菜单和接口权限。医生主要访问病例、检查、影像、推理和评价功能；科研人员主要访问数据浏览、模型配置和模型对比功能；管理员主要访问用户管理和日志审计功能。

权限控制不仅体现在前端菜单隐藏，也在后端接口层进行校验，防止用户通过直接调用接口实现越权访问。

3. 数据隔离设计

系统在业务层和数据访问层实现数据隔离。医生默认只能查看和操作自己创建或被授权范围内的病例、检查和影像；科研人员可查看用于科研分析的数据和模型结果；管理员可管理用户和查看日志，但不直接参与医生业务数据修改。

数据隔离逻辑由后端统一控制，而不是仅依赖前端页面限制。对于病例、影像、任务和结果等关键数据，后端在查询和修改前均需要校验当前用户角色与数据归属关系，保证不同角色的数据访问边界清晰。

4. 异步推理任务设计

AI 推理任务可能耗时较长，如果采用同步请求方式，前端页面需要一直等待推理完成，用户体验较差。系统通过 RabbitMQ 将推理任务异步化：用户提交推理请求后，后端立即创建任务记录，并将任务投递到消息队列中，前端可以通过任务 ID 查询状态。

任务状态包括待处理、处理中、成功、失败等。后台消费者从队列中取出任务，调用 FastAPI 推理服务完成检测，并将结果写入数据库。推理失败时，系统记录失败原因，并可通过 DLQ 死信队列对异常任务进行隔离和后续处理。

该设计避免了长时间 HTTP 连接占用，提高了系统在批量推理场景下的稳定性和可用性。

5. Redis 结果缓存设计

医疗影像系统中，同一张影像的推理结果可能会被医生复核、科研人员分析或多次导出。如果每次访问都重新调用 AI 服务，会造成重复计算和性能浪费。

系统引入 Redis 作为结果缓存。对于同一影像、同一模型和相同参数组合的推理结果，系统可以优先读取缓存数据，减少重复访问数据库和 AI 服务的开销。缓存机制能够提升结果查看速度，降低 AI 推理服务压力，尤其适用于历史结果查看和科研端批量浏览场景。

6. 多模型统一封装设计

系统当前支持 YOLOv11 和 RT-DETR 两类目标检测模型。不同模型的内部结构、输入格式和后处理方式存在差异，但系统通过 FastAPI 推理服务进行统一封装，对外提供一致的推理接口和结果格式。

统一结果格式包括模型编号、检测类别、置信度、检测框坐标、原图尺寸、推理耗时等信息。前端和后端业务层无需关心模型内部实现，只需要按照统一格式处理结果。

该设计使系统具备良好的模型扩展能力。后续如果需要加入新的目标检测模型，只需在 AI 推理层增加对应 Engine，并在模型配置中注册即可。

7. DICOM 解析与智能导入设计

系统支持 DICOM 文件解析和智能导入。DICOM 文件中包含影像像素数据和大量元数据，例如患者信息、检查时间、检查描述、设备信息等。系统通过后端解析 DICOM 文件，将元数据转换为结构化 JSON，并将像素数据转换为可预览图片。

在智能导入场景下，系统根据 DICOM 元数据自动生成病例、检查和影像的候选信息。用户可以选择创建新病例，也可以选择挂载到已有病例。正式导入前，系统提供人工审查与修改入口，用户确认后再写入数据库。

该设计降低了医生手动录入病例和检查信息的成本，也提高了 DICOM 数据导入的准确性和可控性。

8. 结果可视化与导出设计

AI 推理服务返回检测框、类别和置信度后，后端将结果保存到数据库，前端在影像详情页将检测框叠加显示在原图上。系统支持图像缩放查看，并保证检测框位置与原图坐标保持一致。

系统还支持多种结果导出方式，包括结构化 JSON、标注图像和 PDF 报告。JSON 适合科研分析和数据交换，标注图像适合展示检测位置，PDF 报告适合存档、汇报和会诊讨论。导出操作会被记录到日志中，保证结果交付过程可追踪。

9. 可靠性与异常隔离设计

系统将主业务服务和 AI 推理服务分离部署。AI 服务异常、模型加载失败或推理超时不会导致整个业务系统崩溃。后端可以捕获异常，将对应推理任务标记为失败，并向前端返回友好的错误提示。

RabbitMQ 的消息队列机制可以削峰填谷，避免大量推理请求直接压垮 AI 服务。DLQ 死信队列用于隔离异常任务，防止失败消息反复阻塞正常任务处理。Redis、RabbitMQ 或 AI 服务不可用时，系统应尽量保证病例管理、影像浏览、用户管理等核心业务仍可访问。

10. 性能优化设计

系统从前端、后端和基础设施多个层面进行性能优化。

前端页面采用分页加载和条件检索，避免一次性加载过多病例、影像和任务数据。影像列表和详情页根据业务需要加载对应资源，减少无效请求。

后端针对高频查询字段建立索引，例如病例编号、影像 ID、创建时间、任务状态、用户 ID 等，提高列表查询和条件筛选速度。数据库连接池用于减少频繁建立连接的开销。

Redis 缓存用于提升重复结果访问速度，RabbitMQ 异步队列用于避免推理任务阻塞页面。文件系统用于存储原图和标注图，数据库仅保存路径和元数据，避免大文件直接进入数据库造成性能问题。

11. 安全性设计

系统在认证、授权、数据、文件和日志等方面进行安全控制。

密码采用 BCrypt 等安全哈希方式存储，避免明文保存。用户登录后通过 JWT 进行身份认证，后端接口通过 Spring Security 和 RBAC 进行权限校验。敏感字段如患者姓名、患者编号等在列表展示时进行脱敏处理，降低隐私泄露风险。

文件上传采用类型白名单限制，仅允许 DCM、PNG、JPG/JPEG 等受支持格式，并对文件大小进行限制。文件存储时使用系统生成的文件名，避免路径遍历和文件覆盖风险。

关键操作如登录、上传、DICOM 导入、发起推理、医生评价、结果导出、模型参数修改等均记录日志，便于后续审计和问题追踪。

12. 权限控制与数据隔离设计

系统采用 JWT + Spring Security + RBAC 的权限控制方案。用户登录成功后，前端在后续请求中携带 JWT，后端通过 JwtAuthFilter 解析用户身份，并根据 SecurityConfig 中配置的访问规则进行接口级权限校验。

系统将用户划分为管理员、医生、科研人员三类角色。管理员主要访问用户管理和日志审计接口；医生主要访问病例、检查、影像、DICOM 导入、推理任务、医生评价和结果导出等业务接口；科研人员主要访问影像浏览、推理结果查看、模型配置和多模型对比相关接口。

除角色权限外，系统还在业务层进行数据归属校验。医生默认只能访问自己创建或被授权的病例、检查和影像；科研人员仅能访问用于科研分析的数据范围；管理员负责用户和日志管理。该设计避免了用户通过直接调用接口绕过前端菜单限制，从而保证系统的数据隔离和访问安全。

13. 可扩展性设计

系统在功能和技术两方面均保留扩展空间。

业务层采用模块化设计，新增功能可以通过增加 Controller、Service 和 Repository 模块实现，不需要破坏已有结构。AI 推理层采用模型 Engine 方式封装，便于后续新增模型或替换现有模型。

模型参数通过配置管理，科研人员可以在前端调整 conf、iou、max_det、top_k 等参数，减少因参数调整带来的重新部署成本。异步任务、结果缓存和文件存储等基础能力也可以复用于后续批量分析、模型评估、报告生成和数据集构建等扩展功能。

14. 其他设计考虑

开发语言与框架如下：

前端：React + Vite + JavaScript + CSS

后端：Spring Boot + Spring Security + JWT + JPA/Repository

AI 服务：Python + FastAPI + YOLOv11 + RT-DETR

数据库：MySQL

缓存：Redis

消息队列：RabbitMQ

文件存储：服务器文件系统

接口格式：HTTP REST + JSON

接口文档：Swagger / OpenAPI

测试工具：Postman / Apifox / DBeaver

版本控制：Git + GitHub

系统遵循前后端分离、接口契约明确、配置与代码分离、核心业务分层实现的设计原则。整体架构既满足当前课程项目的功能验收要求，也为后续模型升级、功能扩展和部署优化保留了空间。

3.2 用户界面设计

用户界面设计图：

医生界面：

<https://www.figma.com/make/D9YzIxp0joSCSDP4Cz5M2H/Hospital-Data-Management-Dashboard---Community---Copy---Community-?t=e8wY0XWh3284cPF4-1>

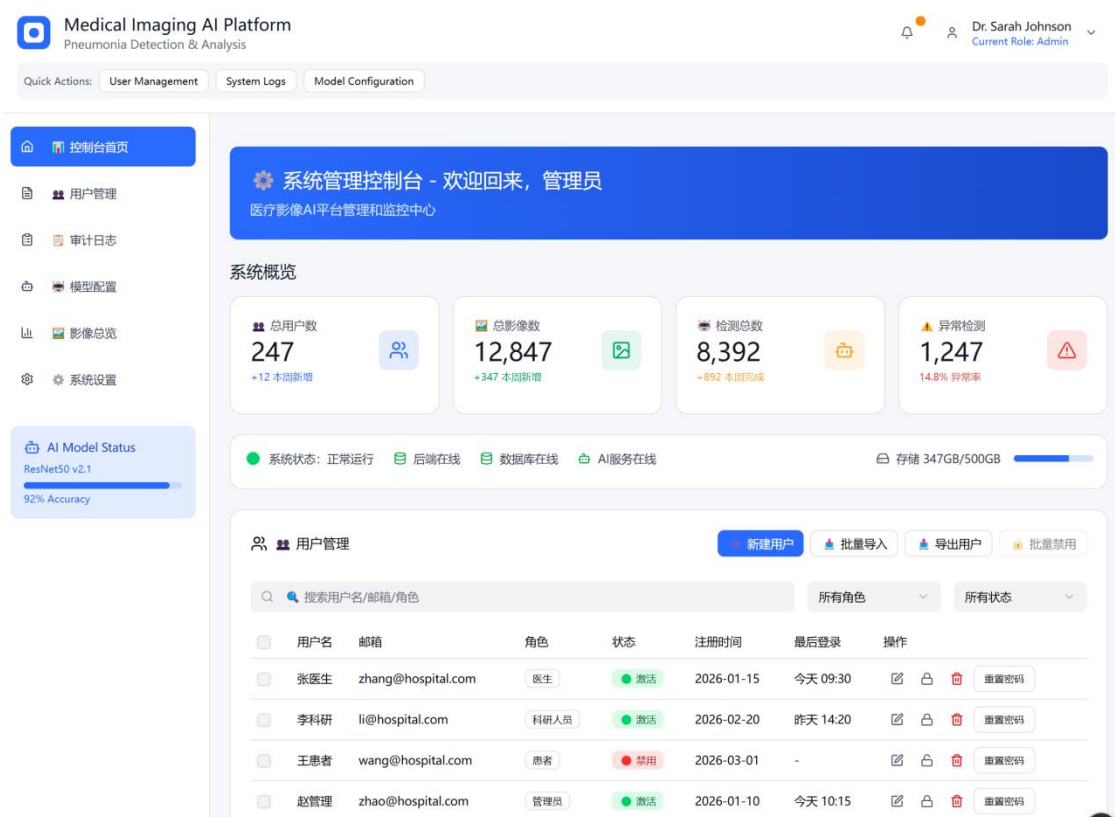
管理员界面：

<https://www.figma.com/make/56n8j29zns6mAtrJhr7fJe/Hospital-Data-Management-Dashboard---adminview---Copy-?t=WmhuiQRJgvZGveSy-1>

科研人员界面：

<https://www.figma.com/make/eO2clHb7we1XFwXPt935bR/Hospital-Data-Management-Dashboard---researcherview?t=SX2JvgORRBHqSziG-1>

示意图：



3.3 接口设计

本系统采用前后端分离架构，前端通过 HTTP/JSON 与 Spring Boot 后端进行通信，后端通过 HTTP REST 与 FastAPI AI 推理服务进行内部调用。系统接口遵循 RESTful 风格，使用统一的 URL 路径、请求方法和 JSON 数据格式组织业务交互。

系统接口文档通过 Swagger/OpenAPI 自动生成，便于前端开发、后端联调和测试人员进行接口验证。当前后端接口文档为“医疗影像数据管理平台 API”，主要覆盖用户认证、用户管理、病例管理、检查管理、影像管理、DICOM 解析与导入、推理任务、结果导出和模型配置等模块；AI 推理服务接口文档为

“PneumoVision Internal Inference Service”，主要覆盖单张推理、批量推理、模型列表、模型参数更新和 DICOM 转 PNG 等内部推理能力。

接口链接：

[医疗影像数据管理平台 API](#)

3.3.1 接口设计原则

系统接口设计遵循以下原则：

1. 前后端分离原则

前端仅负责页面展示和用户交互，通过 API 获取数据和提交操作；后端负责业务逻辑、权限控制、数据校验和持久化处理。

2. RESTful 风格原则

系统使用 GET、POST、PUT、DELETE 等 HTTP 方法表达查询、新增、修改和删除操作。例如，病例资源使用 /api/cases 进行统一管理，单个病例使用 /api/cases/{id} 访问。

3. 统一响应格式原则

后端接口统一返回 ApiResponse 结构，包含业务状态码、提示信息和数据内容，便于前端统一处理成功、失败和异常提示。

4. 统一认证原则

除登录、初始化管理员等公开接口外，业务接口均需要携带 JWT 令牌访问。后端通过 Spring Security 和 JWT 过滤器完成身份认证，并结合 RBAC 进行角色权限校验。

5. 模块化接口组织原则

接口按照业务模块进行划分，包括认证模块、管理员模块、病例模块、检查模块、影像模块、DICOM 模块、推理任务模块、模型配置模块等，便

于维护和扩展。

6. 内部服务隔离原则

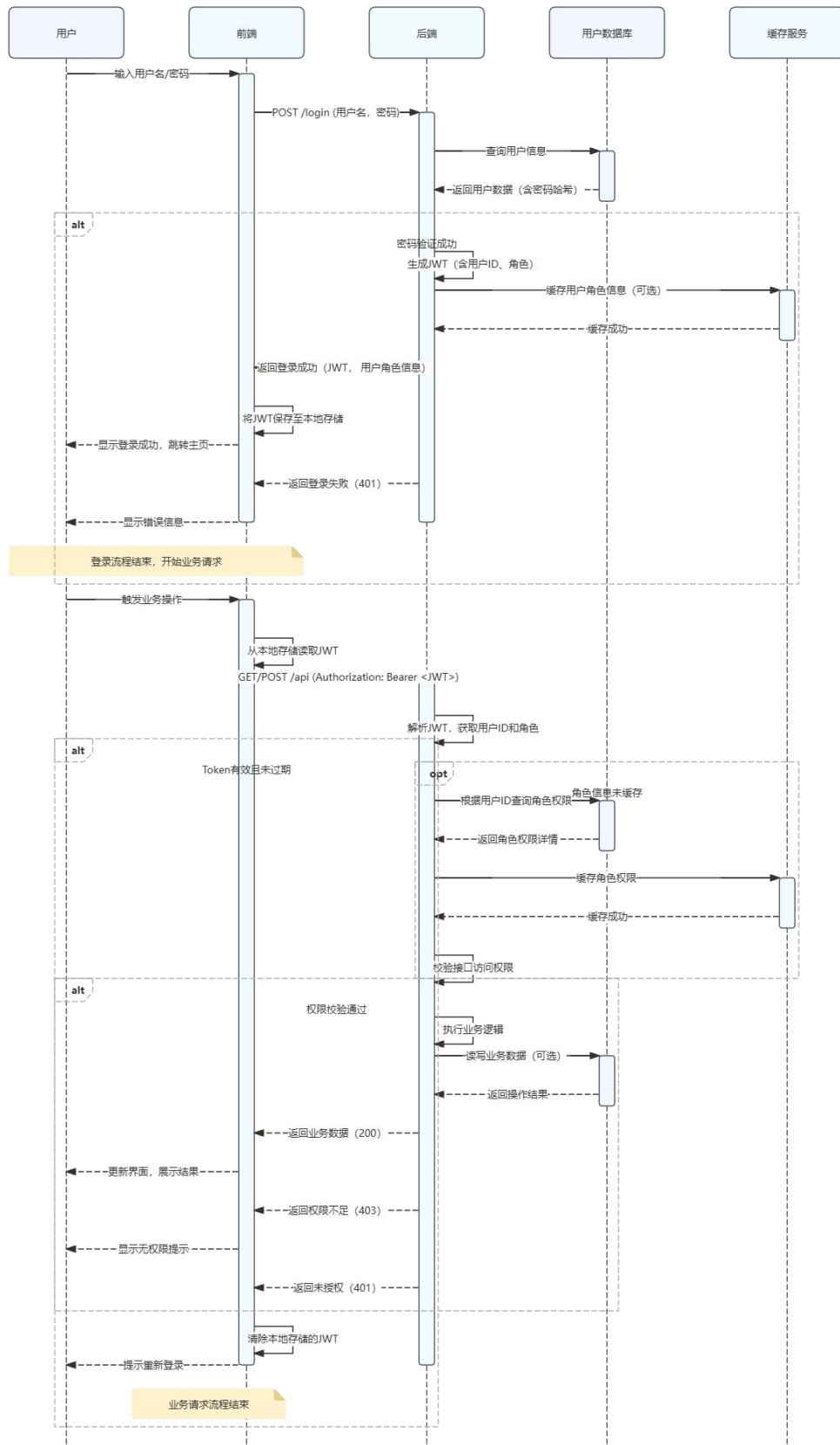
FastAPI AI 推理服务主要作为内部服务供 Spring Boot 后端调用，前端不直接访问推理服务。这样可以减少前端与 AI 服务的耦合，并由后端统一处理权限、任务状态和结果存储。

3.3.2 接口访问与认证方式

系统采用 JWT 进行身份认证。用户登录成功后，后端返回令牌，前端在后续请求中通过请求头携带该令牌。

Authorization: Bearer <token>

接口访问流程如下：



当用户未登录、令牌失效或访问越权接口时，系统返回 401 或 403 类型错误，

并由前端提示用户重新登录或无权访问。

3.3.3 后端核心接口分类

接口模块	典型接口	请求方式	说明
认证模块	/api/auth/login	POST	用户登录, 返回 JWT 和用户角色信息
认证模块	/api/auth/logout	POST	用户退出登录
管理员模块	/api/admin/users	GET / POST	查询用户列表、创建用户
管理员模块	/api/admin/users/{id}/enable	POST	启用指定用户
管理员模块	/api/admin/users/{id}/disable	POST	禁用指定用户
病例模块	/api/cases	GET / POST	查询病例列表、创建病例
病例模块	/api/cases/{id}	GET / PUT / DELETE	查看、修改、删除指定病例
检查模块	/api/cases/{caseId}/studies	GET / POST	查询病例下检查、创建检查
检查模块	/api/cases/{caseId}/studies/{studyId}	GET / PUT / DELETE	查看、修改、删除指定检查
影像模块	/api/images/upload	POST	上传单张影像
影像模块	/api/images/upload-batch	POST	批量上传影像

块			像
影像模块	/api/images	GET	按条件分页查询影像
影像模块	/api/images/{id}/preview	GET	获取影像预览图
DICOM模块	/api/dicom/parse	POST	解析 DICOM 文件，返回元数据和预览信息
DICOM模块	/api/dicom/import	POST	DICOM 智能导入，创建或挂载病例/检查/影像
DICOM模块	/api/dicom/export	POST	导出 DICOM 解析结果
推理任务模块	/api/inference/tasks	GET / POST	查询推理任务、创建单个推理任务
推理任务模块	/api/inference/tasks/batch	POST	批量创建推理任务
推理任务模块	/api/inference/tasks/{taskId}	GET	查看指定推理任务详情
推理结果模块	/api/inference/tasks/{taskId}/results	GET / PUT	查询或保存推理结果
医生评价模块	/api/inference/tasks/{taskId}/review	POST	医生提交 AI 结果评价
结果导出模块	/api/inference/tasks/{taskId}/export	GET	导出结构化推理结果
结果导出模块	/api/inference/tasks/{taskId}/export-pdf	GET	导出 PDF 检测报告
结果可	/api/inference/tasks/{taskId}/overlay-preview	GET	获取带检测

视 化 模 块			框的结果预览图
模 型 配 置模块	/api/models	GET	查询模型配置列表
模 型 配 置模块	/api/models/{modelId}/params	PUT	修改模型默认参数
模 型 配 置模块	/api/models/{modelId}/enable	PUT	启用模型
模 型 配 置模块	/api/models/{modelId}/disable	PUT	禁用模型

3.3.4 AI 推理服务内部接口

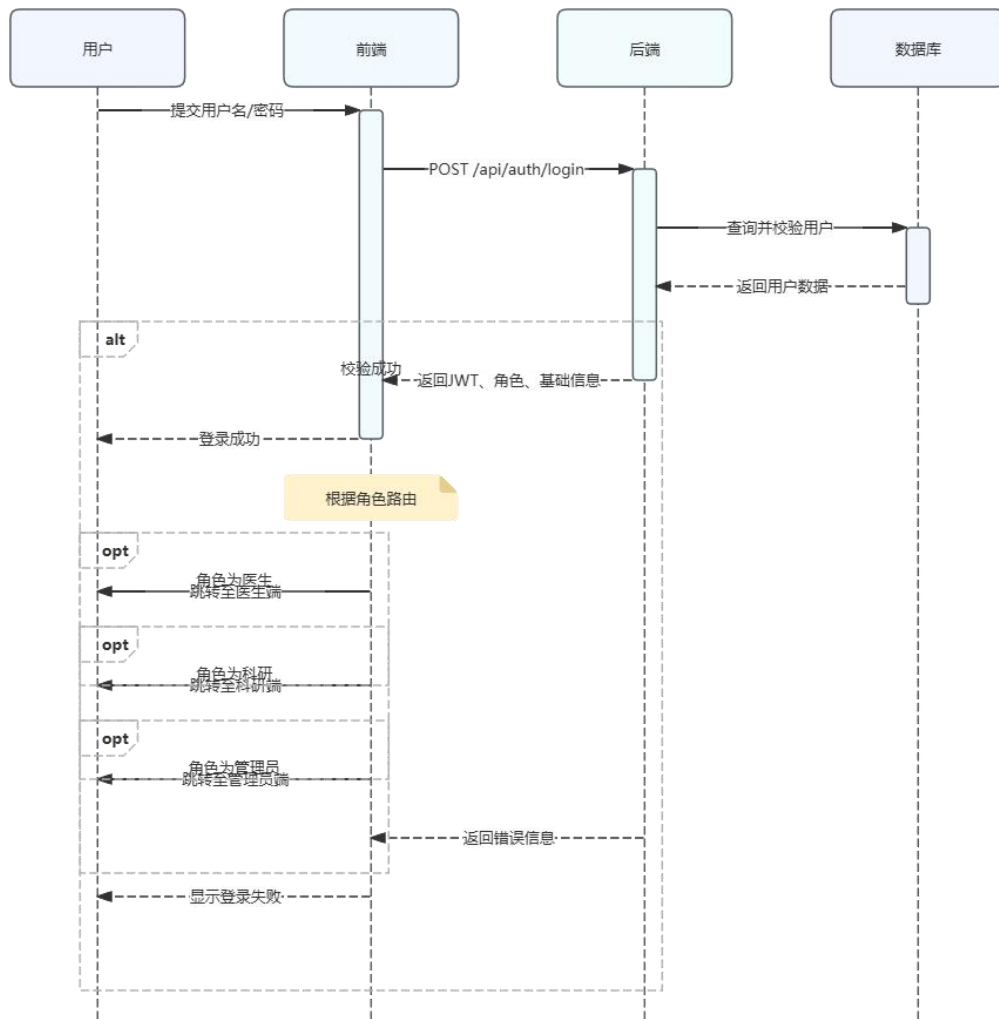
AI 推理服务由 FastAPI 提供，主要供 Spring Boot 后端内部调用。该服务不直接暴露给普通前端用户，避免绕过业务系统的认证、权限和任务管理逻辑。

接口模块	典型接口	请求方式	说明
单张推理	/api/inference/tasks/predict	POST	对单张影像执行 AI 推理
批量推理	/api/inference/tasks/batch-predict	POST	对多张影像执行批量推理
模型列表	/api/models	GET	获取当前 AI 服务中的模型配置
模型参数	/api/models/{model_id}/params	PUT	更新指定模型的默认推理参数
模型状态	/api/models/{model_id}/enable	PUT	启用指定模型
模型状态	/api/models/{model_id}/disable	PUT	禁用指定模型
DICOM 转换	/api/dicom/convert-to-png	POST	将 DICOM 文件转换为 PNG 图片

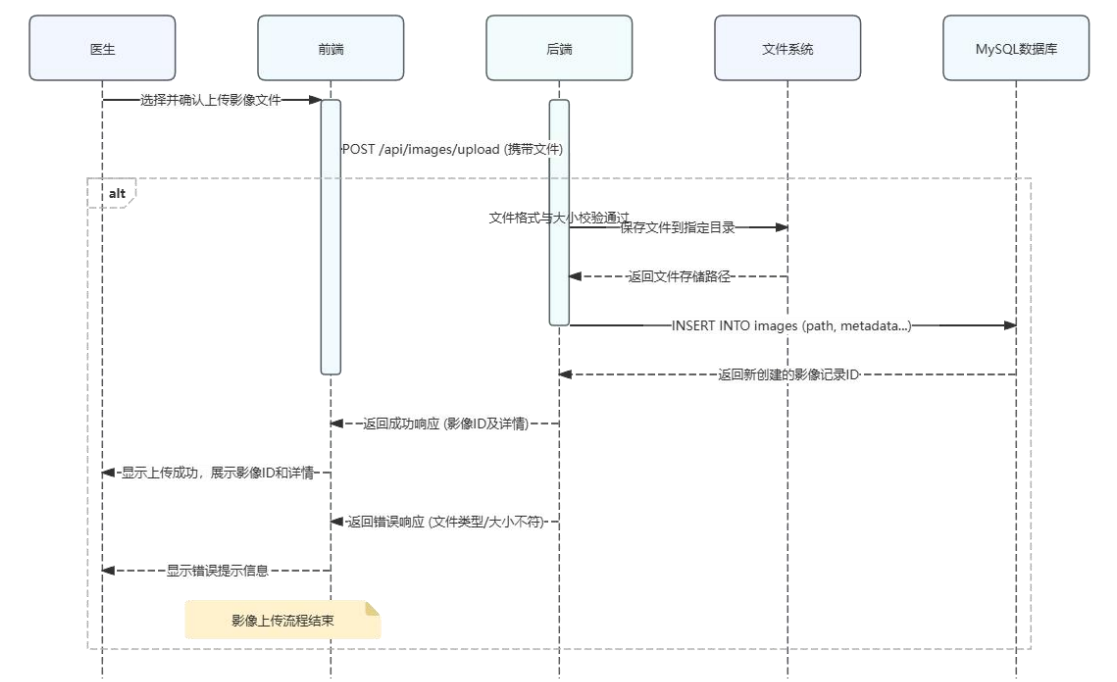
AI 推理服务的请求通常采用 multipart/form-data 格式上传影像文件，返回统一的 JSON 推理结果。模型推理结果包括模型编号、检测类别、置信度、检测框坐标、原图尺寸和推理耗时等信息。Spring Boot 后端接收到结果后，会将其转换为系统内部的推理结果对象，并保存到数据库中。

3.3.5 典型接口调用流程

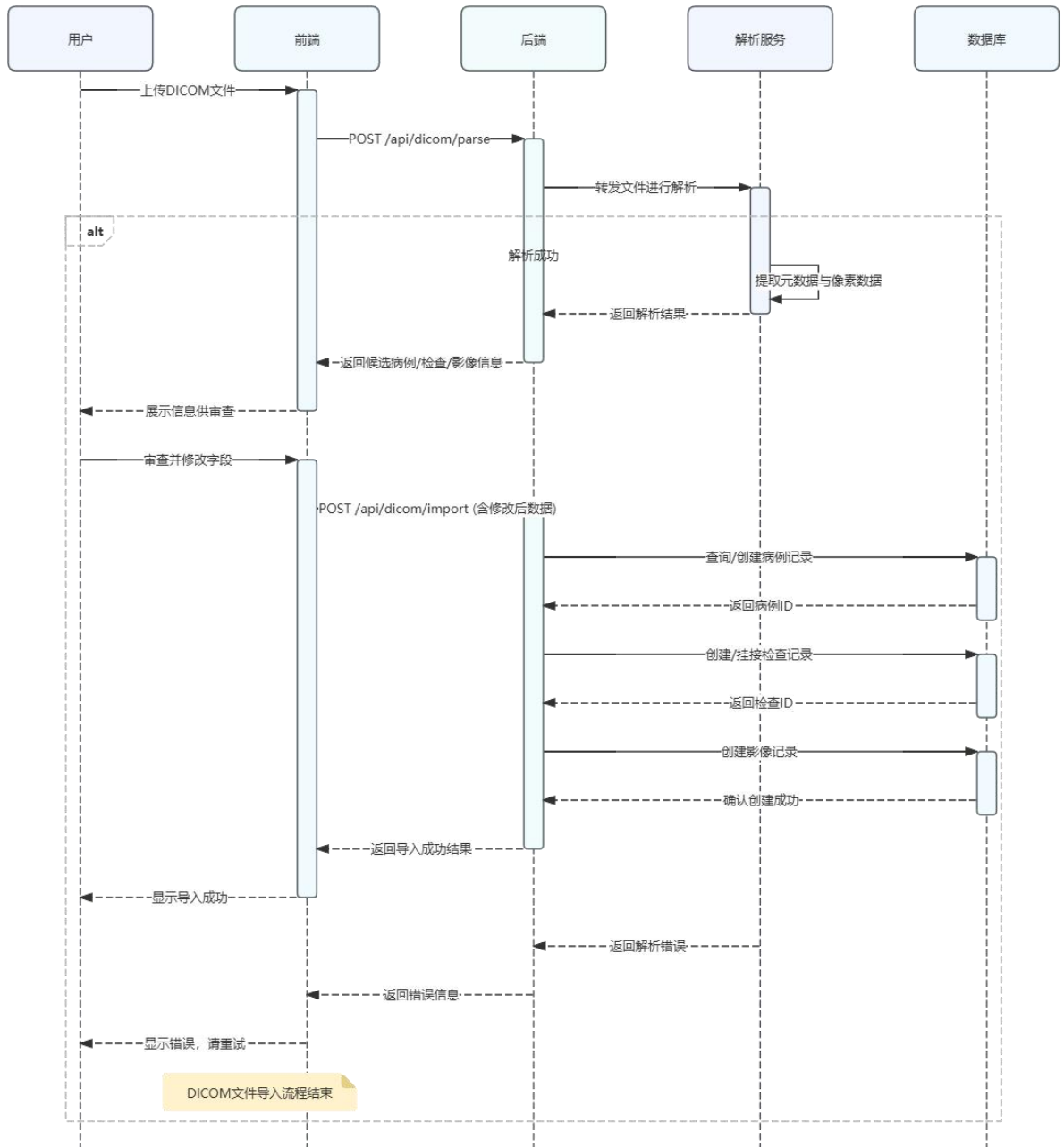
1. 登录认证流程



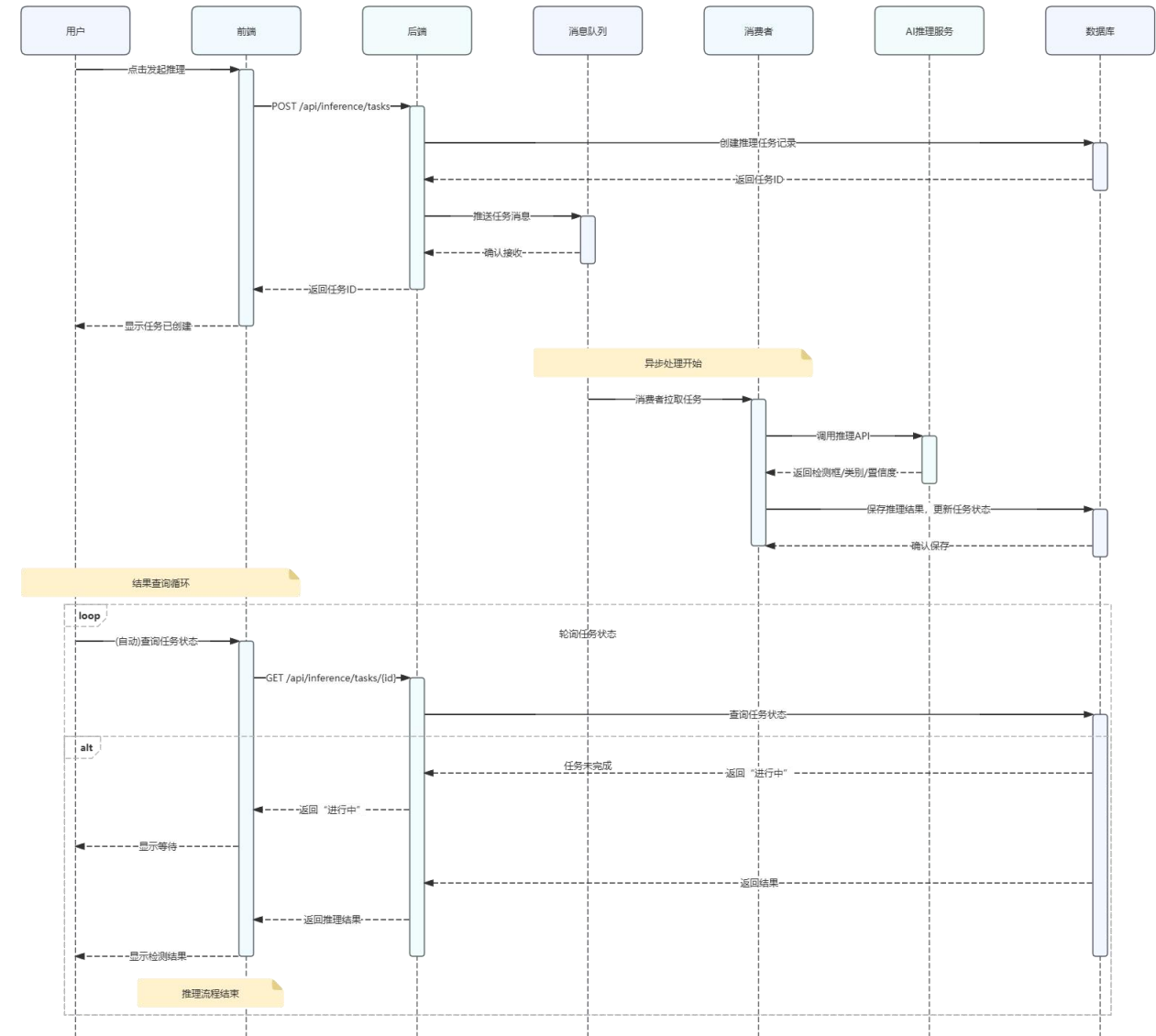
2. 影像上传流程



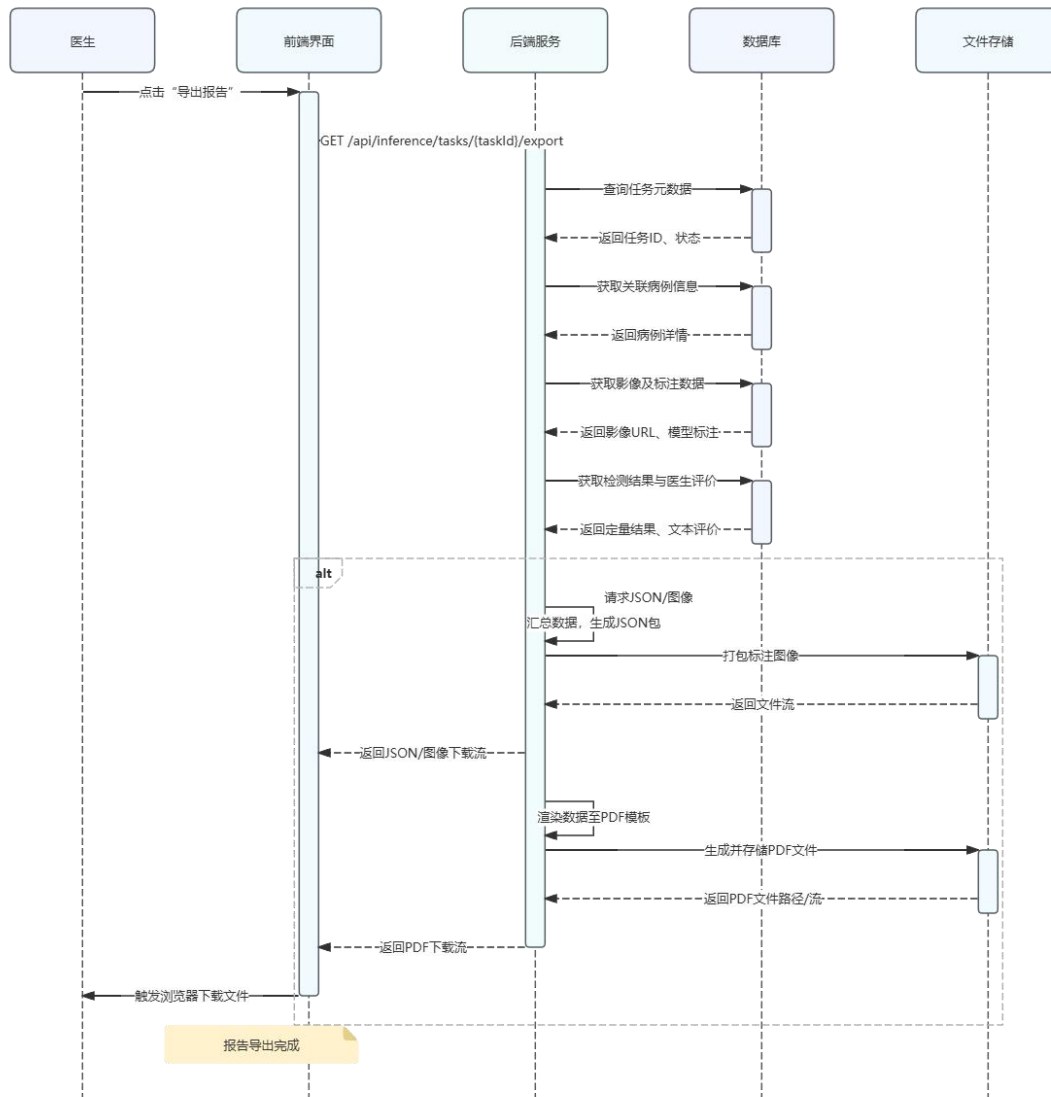
3. DICOM 智能导入流程



4. AI 推理任务流程



5. 结果导出流程

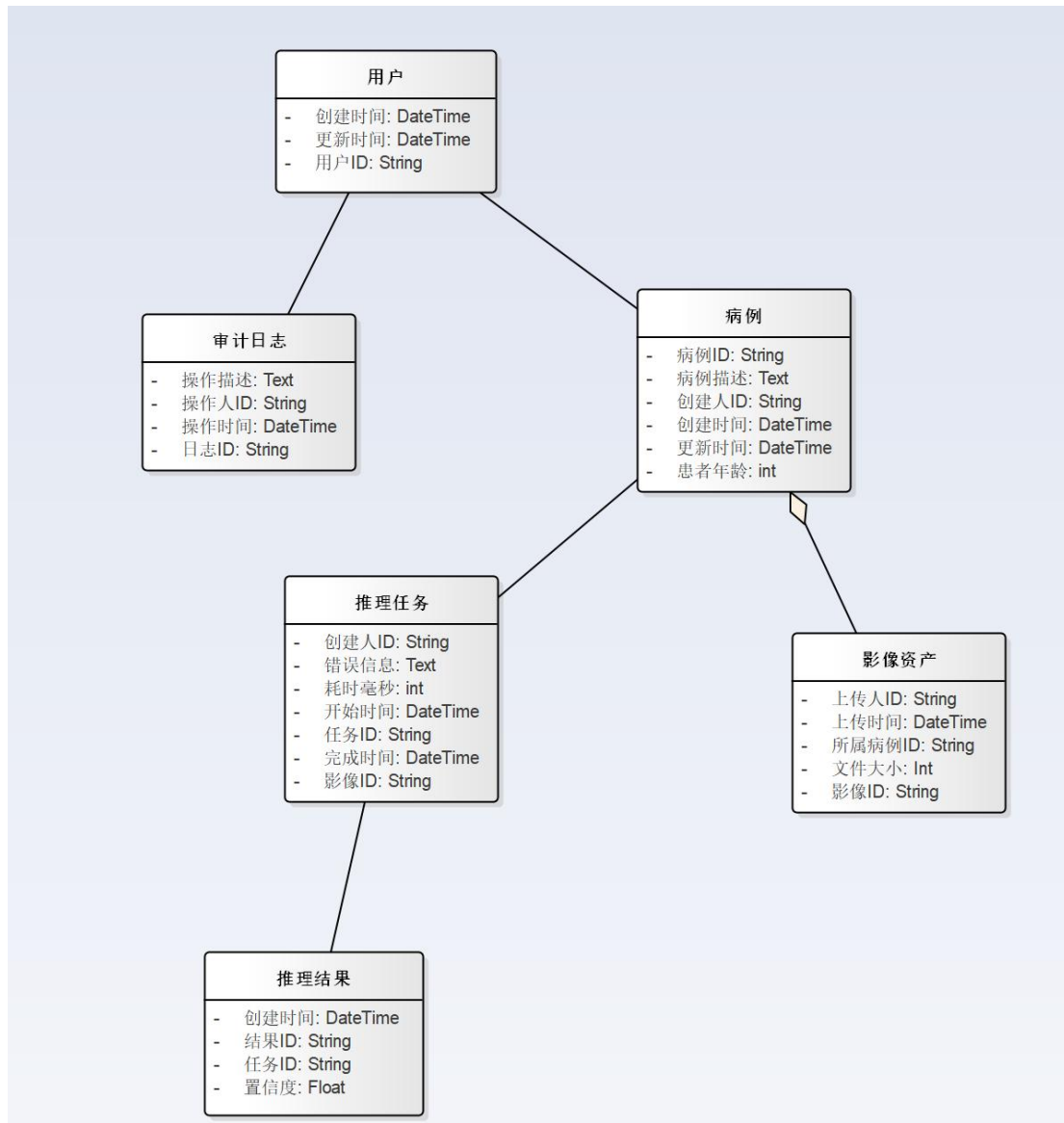


3.3.6 接口安全与异常处理

系统接口通过以下方式保证安全性与稳定性：

1. 业务接口默认需要 JWT 认证；
2. 后端基于用户角色进行接口权限控制；
3. 医生、科研人员和管理员访问不同接口时，后端会校验角色权限；
4. 对病例、影像、推理任务等数据，后端会进一步校验数据归属，防止越权访问；
5. 文件上传接口限制文件类型和文件大小；
6. AI 推理失败、DICOM 解析失败、文件导出失败等异常均返回明确错误信息；
7. 关键操作记录日志，便于后续审计和问题排查。

3.4 数据设计



3.5 部署设计

